

# MEMORY ORGANIZATION

# MEMORY ORGANIZATION

**Basic Concepts, Memory Hierarchy and it's need**

**Parameters used to Measure the Performance**

**Types of Memory Components**

**Semiconductor RAM Memories**

**Memory Module Design**

**ROM**

**Cache Memories**

**Mapping Functions**

**Replacement Algorithms**

**Memory Interleaving**

**Memory Performance Consideration**

**Virtual Memory Organization**

# Memory

## Some Basic Concepts

- What is the max. size of memory?
- Address space
  - 16-bit :  $2^{16} = 64\text{K}$  memory locations
  - 32-bit :  $2^{32} = 4\text{G}$  memory locations
  - 40-bit :  $2^{40} = 1\text{ T}$  memory locations
- Ideally,
  1. Fast
  2. Large
  3. Inexpensive
- Is it possible to meet all 3 requirements simultaneously ?

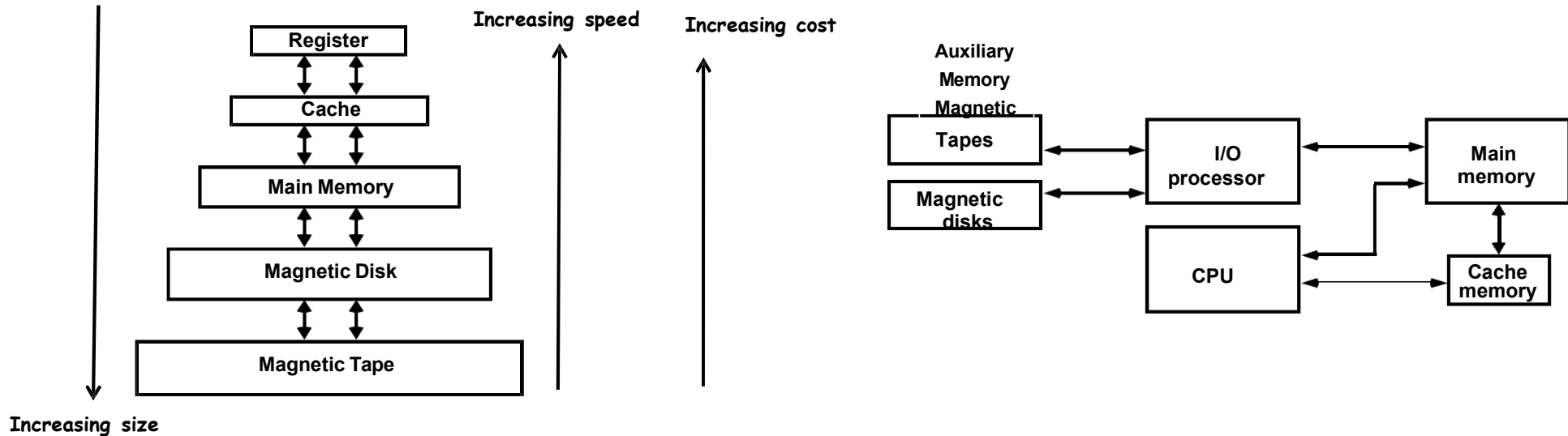
# Introduction

- **Even a sophisticated processor may perform well below an ordinary processor:**
  - Unless supported by matching performance by the memory system.
- **The focus of this module:**
  - Study how memory system performance has been enhanced through various innovations and optimizations.

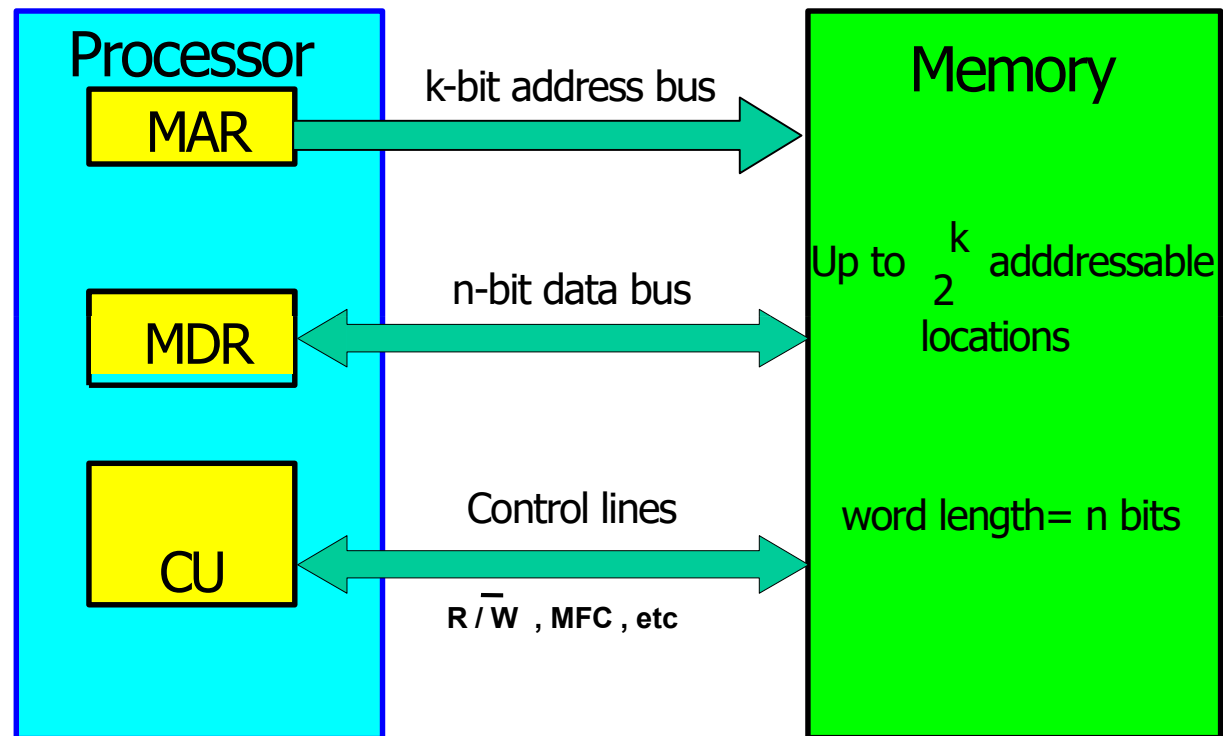
# MEMORY HIERARCHY

*Memory Hierarchy*

Memory Hierarchy is to obtain the highest possible access speed while minimizing the total cost of the memory system

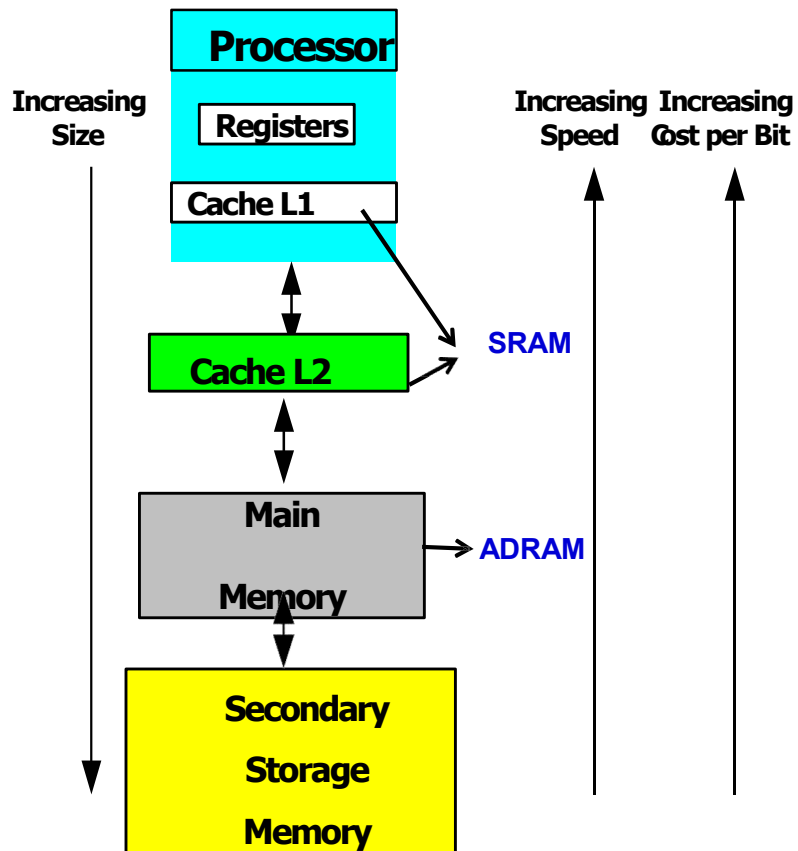


# Basic Concepts of Memory



Connection of the memory to the processor

# Basic Concepts of Memory



1. Fastest access is to the data held in Processor Registers. Registers are at the top of the Memory Hierarchy.
2. Relatively small amount of memory that can be implemented on processor chip. This is cache.
3. Two levels of cache. Level 1 (L1) Cache is on the Processor Chip
4. Level 2 (L2) Cache is in between Main Memory and Processor.
5. Next level is Main Memory
6. Much larger, but much slower than Cache Memory.
7. Next level is Magnetic Disks. Huge amount of inexpensive storage.
8. Speed of Memory access is critical, the idea is to bring instructions and data that will be used in the near future as close to the processor as possible.

# Basic Concepts of Memory

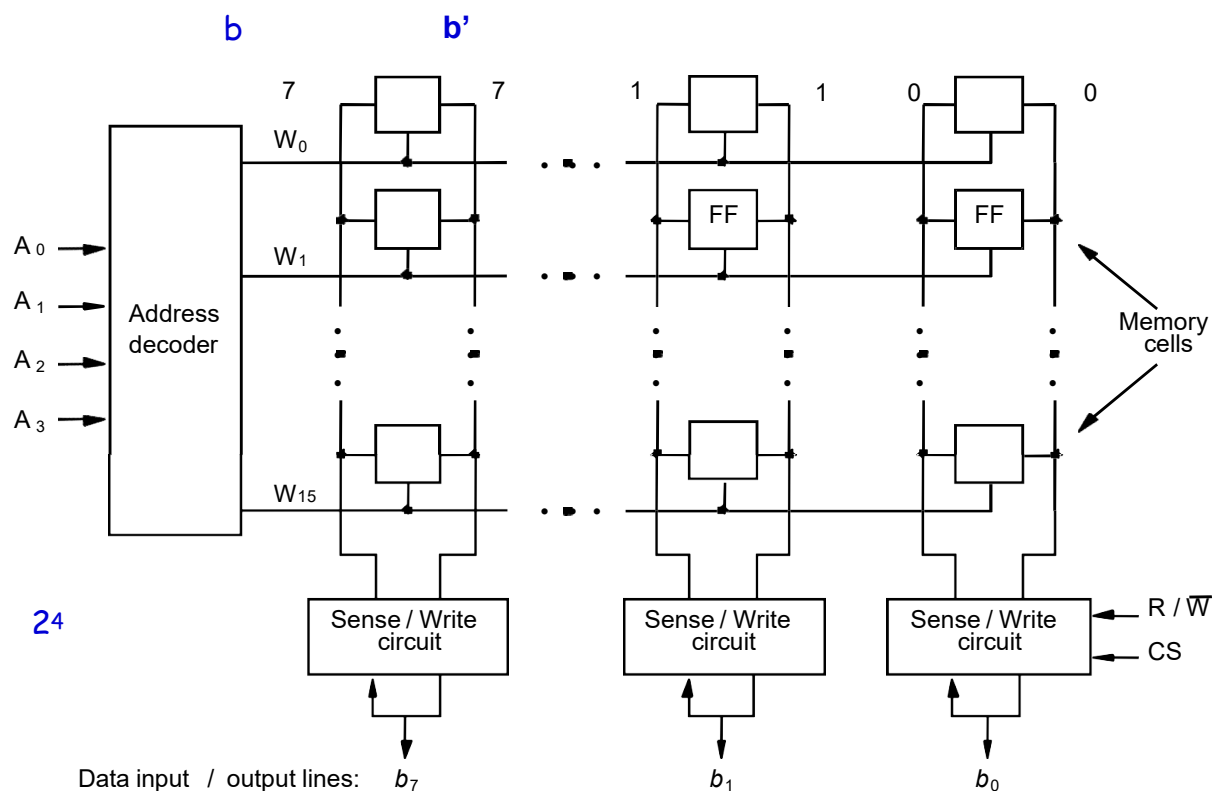
- **Random Access Memory** → **any location can be accessed for Read / Write operation in fixed amount of time .**
- **Types of RAM :→**
  1. **Static Memory / SRAM : Capable of retaining states as long as power is applied, volatile in nature.[High Cost & Speed]**
  2. **Asynchronous DRAM : Dynamic RAM are less expensive but they do not retain their state indefinitely. Widely used in computers.**
  3. **Synchronous DRAM : Whose operation is directly Synchronized with a clock signal.**
  4. **Performance Parameter :- Bandwidth & Latency.**
  5. **Bandwidth :-Number of bytes transfer in 1 unit of time.**
  6. **Latency:- Amount of time takes to transfer a word of data to & from memory.**



# Basic Concepts of Memory

- **Read Only Memory / ROM** : → location can be accessed for read operation only in fixed amount of time. Capable of retain states called as, non-volatile in nature.
  - **Programmable ROM** : Allows data to be loaded by user.
  - **Erasable PROM** : Erased [ by UV ray] Stored data to load new data.
  - **Electrically EPROM** : Erased by different voltages.
- **Memory uses semiconductor integrated circuit to increase performance.**
- **To reduce Memory Cycle Time → Use Cache Memory → A small SRAM physically very close to processor which works on locality of reference.**
- **Virtual Memory is used to increase the size of the physical memory.**

# Internal Organization of Memory Chips



Organization of bit cells in a memory chip

- Memory cells are organized in an array [ Row & Column format ] where each cell is capable of storing one bit of information.
- Each row of cell contains memory word / data & all cells are connected to a common word line, which is driven by address decoder on chip.
- Cell in each column are connected to sense / write circuit by 2 bit lines.
- sense / write circuits are connected to data I/O lines of chip.
- READ Operation → Sense / write circuit  
Sense / Read information stored in cells selected by a word line & transmit same information to o/p data line.
- WRITE Operation → Sense / write circuit receive i/p information & store it in the cell.
- If a memory chip consist of 16 memory words of 8 bit each then it is referred as 16 x 8 organization or 128 x 8 bit organization.
- The data I/O of each sense / write circuit are connected to a single bidirectional data line that can be connected to the data bus of a computer.
- 2 control lines Read / Write [Specifies the required operation ] & Chip Select (CS) [ select a chip in a multichip memory ].
- It can store 128 bits & 14 external connections like address, data & control lines.

# Internal Organization of Memory Chip

## An Example

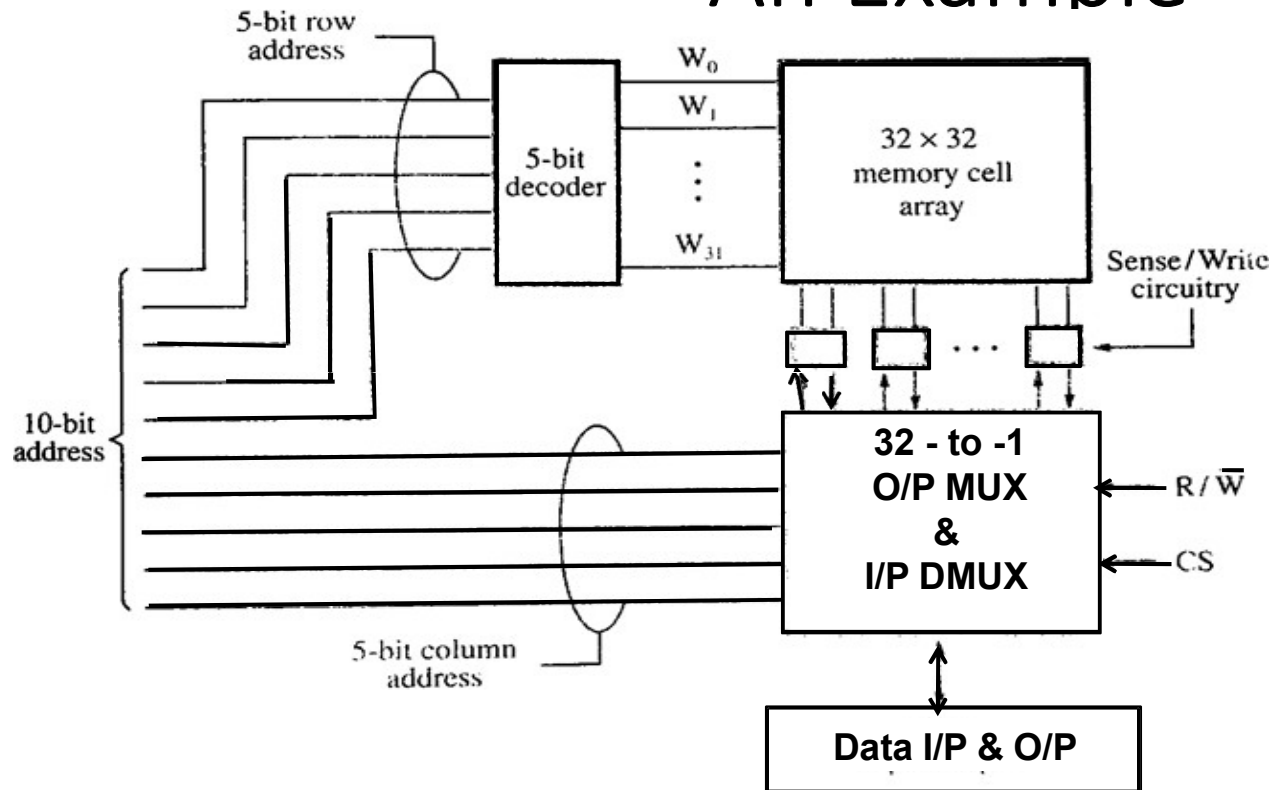


Figure 5.3 Organization of a 1K x 1 memory chip.

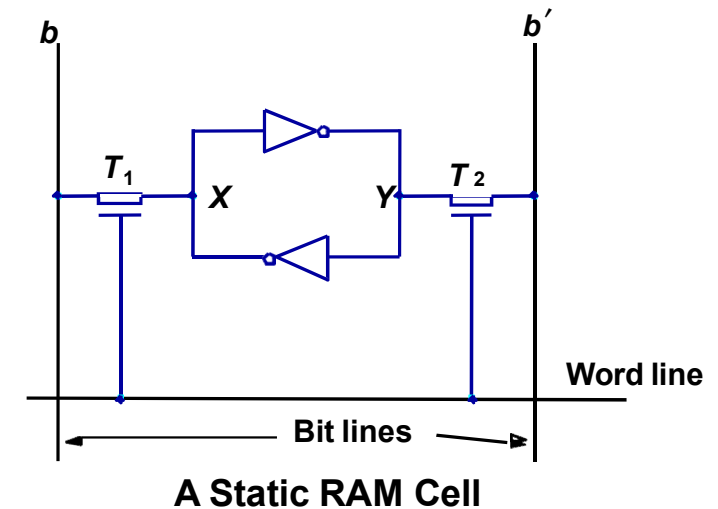
- Design a memory of **1k [ 1024 ]** memory cells.
- For 1k , we require **10 bits address line**.
- So **5 bits** for **rows & columns** each to access address of the memory cell represented in array.
- A row address selects a row of 32 cells, all of which accessed in parallel.
- According to the column address, only one of these cells is connected to the external data line by the **output MUX & input DMUX**.

# Static Memories

- Circuits capable of retaining their state as long as power is applied **Static RAM (SRAM) (volatile)**.
- 2 inverters are cross connected to form a latch.
- Latch is connected to 2 bit lines by transistors  $T_1$  &  $T_2$ .
- Transistors  $T_1$  &  $T_2$  act as switches can be opened & closed under control of word line.
- For ground level transistors turned off (initial time cell is in state 1,  $X=1$  &  $Y=0$  ).

## Read Operation :-

1. Word line activated to close switches  $T_1$  &  $T_2$ .
2. Cell state either 1 or 0 & the signal on bit line b and b' are always complements to each other.
3. Sense / Write circuit set the end value of bit line as output.



## Write Operation :-

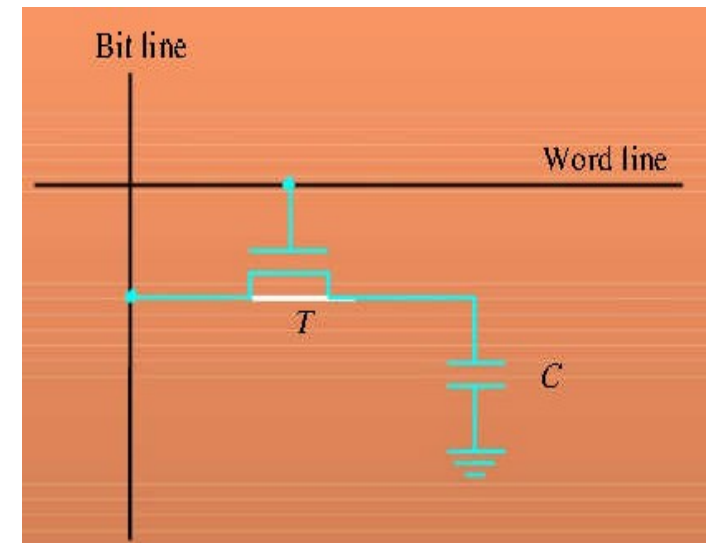
1. State of the cell is set by placing the actual values on bitline b & its complement on b' and activating the word line. [Sense / Write circuit ]

# Asynchronous DRAM

- SRAM's are **fast** but very **costly** due to much number of **transistors** for their cells.
- So, **less expensive cell**, which also **can't retain their state indefinitely** turn into a memory as dynamic RAM [DRAM].
- Data is stored in DRAM cell in form of **charge on capacitor** but only for a period of tens of milliseconds.

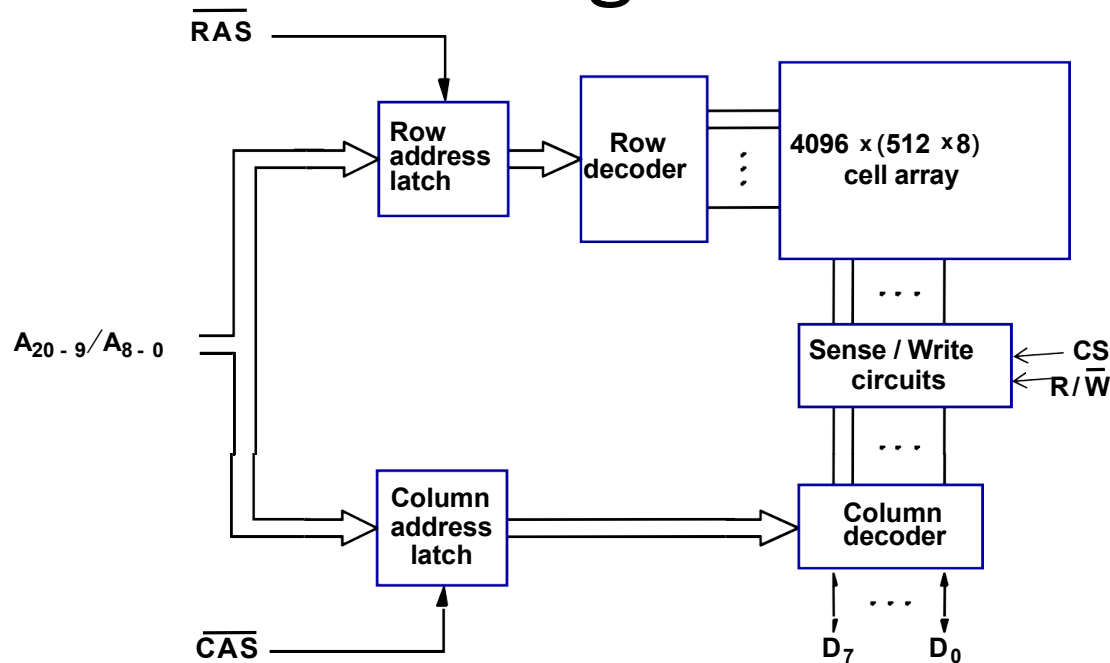
## An Example of DRAM

- DRAM cell consist of a capacitor, **C** , & a transistor, **T** .
- To store information in cell, transistor **T** is turn on, & provide correct amount of **voltage to bit line**.
- After transistor **turn off** capacitor begins to **discharge**.
- So, **Read** operation must be completed before capacitor **drops** voltage below some **threshold value** [ by sense amplifier connected to bit line].



Single Transistor Dynamic memory Cell

# Design 16MB DRAM Chip



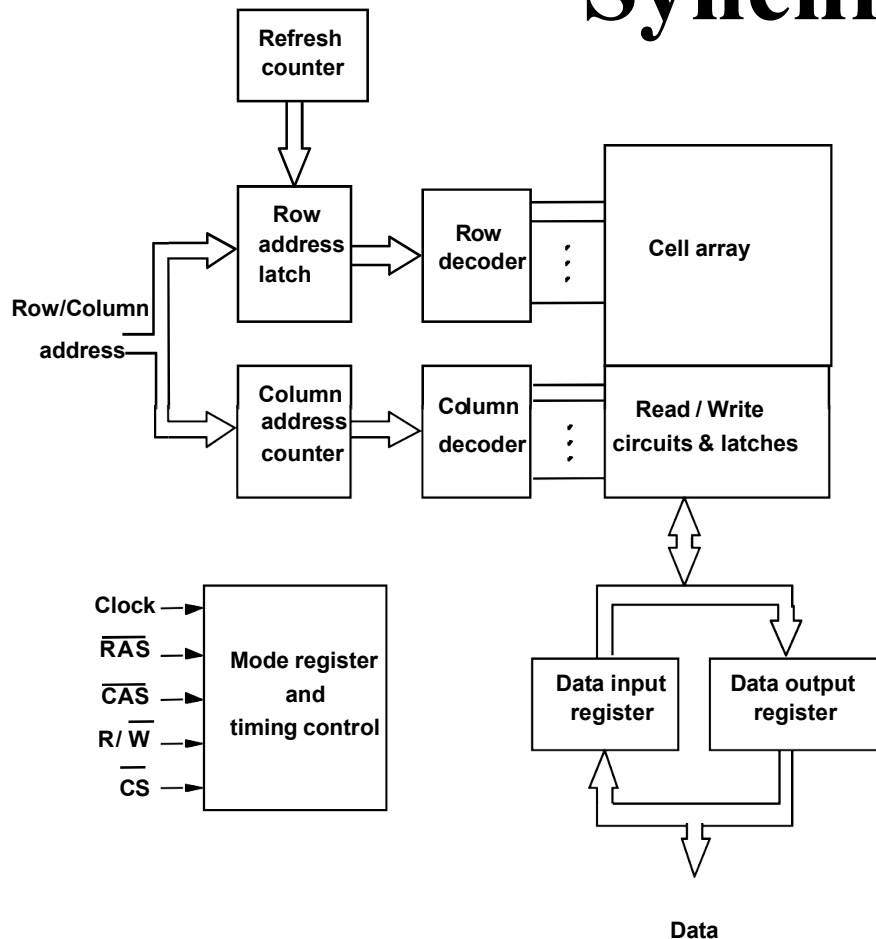
1. Each row can store 512 bytes. 12 bits to select a row, and 9 bits to select a group in a row. Total of 21 bits.
2. First apply the row address, RAS signal latches the row address. Then apply column address, CAS signal latches the address
3. Timing of the memory unit is controlled by a specialized unit which generates RAS and CAS.

- **2 M x 8** memory chip .
- Cells are organized in the form of **4K x 4K** .
- **4096** cells in each row divided into **512** group of 8. Hence **512** byte data can be stored in each row.
- **12** [  $512 \times 8 = 2^{12}$  ] bit address to select row & **9** [  $512 = 2^{12}$  ] bits to specify a group of 8 bits in the selected row.
- **RSA** [Row address strobe] & **CSA** [Column address strobe] will be crossed to find the proper bit to read or write.
- The information on **D7-0** lines is transferred to the selected circuit for write operation.

# Fast Page Mode

- Suppose if we want to access the consecutive bytes in the selected row.
- This can be done without having to **reselect the row**.
  - Add a latch at the output of the sense circuits in each row.
  - All the latches are loaded when the row is selected.
  - Different column addresses can be applied to select and place different bytes on the data lines.
- Consecutive sequence of column addresses can be applied under the control signal **CAS**, without reselecting the row.
  - Allows a block of data to be transferred at a much faster rate than random accesses.
  - A small collection/group of bytes is usually referred to as a block.
- This transfer capability is referred to as the **Fast Page Mode** feature.

# Synchronous DRAM



1. Operation is directly synchronized with processor clock signal.
2. The outputs of the sense circuits are connected to a latch.
3. During a Read operation, the contents of the cells in a row are loaded onto the latches.
4. During a refresh operation, the contents of the cells are refreshed without changing the contents of the latches.
5. Data held in latches correspond to the selected columns are transferred to the output.
6. For a burst mode of operation, successive columns are selected using column address counter and clock.
7. CAS signal need not be generated externally. A new data is placed during raising edge of the clock

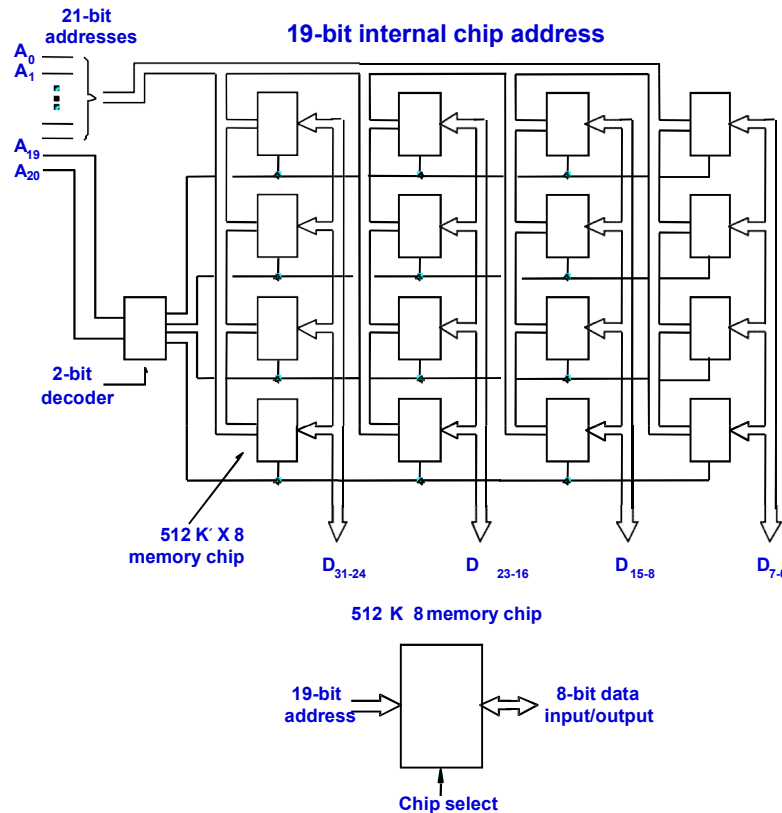


# Double-Data-Rate SDRAM

- In addition to faster circuits, new organizational and operational features make it possible to achieve high data rates during block transfers.
- The key idea is to take advantage of the fact that a large number of bits are accessed at the same time inside the chip when a row address is applied.
- Various techniques are used to transfer these bits quickly to the pins of the chip.
- To make the best use of the available clock speed, data are transferred externally on both the rising and falling edges of the clock. For this reason, memories that use this technique are called *double-data-rate SDRAMs (DDR SDRAMs)*.
- Several versions of DDR chips have been developed. The earliest version is known as DDR. Later versions, called DDR2, DDR3, and DDR4, have enhanced capabilities.

# Structure of Larger Memory

## Static memories

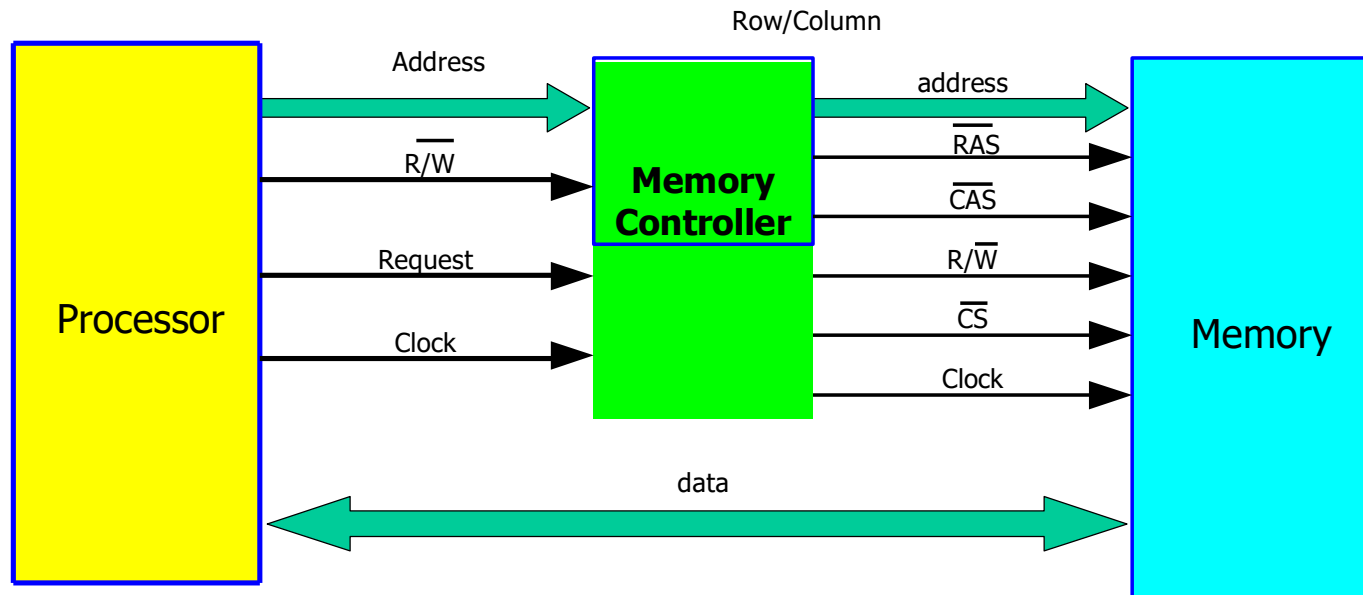


1. Implement a memory unit of 2M words of 32 bits each.
2. Use 512K x 8 static memory chips.
3. Each column consists of 4 chips.
4. Each chip implements one byte position.
5. A chip is selected by setting its chip select control line to 1
6. Selected chip places its data on the data output line, outputs of other chips are in high impedance state.
7. 21 bits to address a 32-bit word.
8. High order 2 bits are needed to select the row, by activating the four Chip Select signals.
9. 19 bits are used to access specific byte locations inside the selected chip.

Organization of 2M X 32 Memory Modules using 512 K x 8 Static Memory Chip

# Memory Controller

- Memory Address are divided into 2 parts.
- High order address bit which select row in the cell array, are provided first & latched into Memory Chip under Control of RAS Signal.
- Low order address bit, which selects a column are provided on the same address & latched through CSA signal.
- However, a processor issues all address bits at the same time.
- In order to achieve multiplexing, Memory Controller circuit is inserted between Processor & Memory.
- Controller accepts a complete address & R/W Signal from Processor under control of REQUEST signal, which indicates Memory Access operation is needed.
- Controller forwards row & Column Address timing to have address multiplexing function.
- Then R/W & CS are sent to Memory.
- Data lines are directly connected between Processor & Memory.



# Read-Only Memory (ROM)

- **Many applications need memory devices to retain contents after the power is turned off.**
  - For example, computer is turned on, the operating system must be loaded from the disk into the memory.
  - Store instructions which would load the OS from the disk.
  - Need to store these instructions so that they will not be lost after the power is turned off.
  - We need to store these instructions into a non-volatile memory.
- **Non-volatile memory is read in the same manner as volatile memory.**
  - Separate writing process is needed to place information in this memory.
  - Normal operation involves only reading of data, this type of memory is called Read-Only memory (ROM).

# Read-Only Memory

## (ROM)

- **Read-Only Memory:**
  - Data are written into a ROM when it is manufactured.
- **Programmable Read-Only Memory (PROM):**
  - Allow the data to be loaded by a user.
  - Process of inserting the data is irreversible.
  - Storing information specific to a user in a ROM is expensive.
- **Erasable Programmable Read-Only Memory (EPROM):**
  - Stored data to be erased and new data to be loaded.
  - Flexibility, useful during the development phase of digital systems.
  - Erasable, reprogrammable ROM.
  - Erasure requires exposing the ROM to UV light.
- **Electrically Erasable Programmable Read-Only Memory (EEPROM):**
  - To erase the contents of EPROMs, they have to be exposed to ultraviolet light.
  - Physically removed from the circuit.
  - EEPROMs the contents can be stored and erased electrically
- **Flash memory:**
  - Has similar approach to EEPROM.
  - Read contents of a single cell, but write contents of an entire block of cells.
  - Higher capacity and low storage cost per bit.
  - Power consumption of flash memory is very low, making it attractive for use in equipment that is battery-driven.

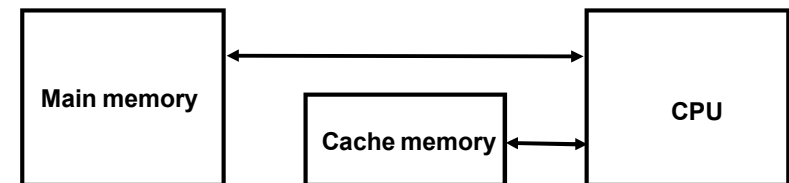
# Cache Memory

- **Processor is much faster than the Main Memory.**
  - As a result, the Processor has to spend much of its time waiting while Instructions and Data are being fetched from the Main Memory.
  - Major obstacle towards achieving good performance.
- **Speed of the Main Memory cannot be increased beyond a certain point.**
- **Cache is an architectural arrangement that makes the Main Memory appear faster to the Processor than it really is.**
- **Relatively small SRAM [ Having low access time ] memory located physically closer to processor.**
- **Cache Memory is based on the property of computer programs known as**  
**“LOCALITY OF REFERENCE”.**
- **Analysis of Programs indicates that many Instructions in localized areas of a Program are executed repeatedly during some period of time, while the others are accessed relatively less frequently.**
  - These instructions may be the ones in a loop, nested loop or few procedures calling each other repeatedly.

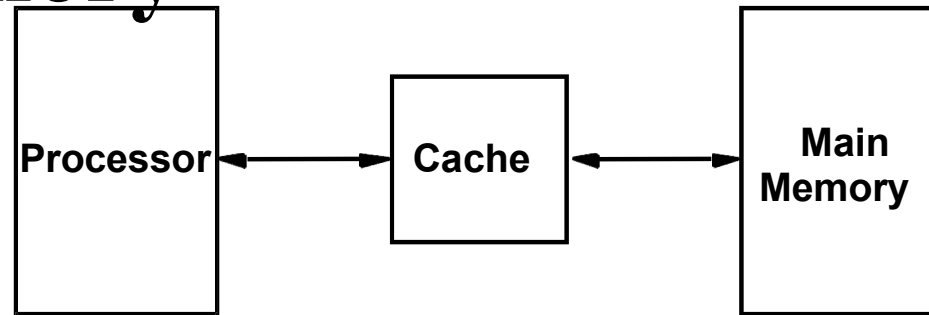
# Cache Memory

## Locality of Reference

- The references to memory at any given time interval tend to be confined within a localized areas.
- This area contains a set of information and the membership changes gradually as time goes by
- **Temporal Locality** →
  - ✓ Recently executed instruction is likely to be executed again very soon.
  - ✓ The information which will be used in near future is likely to be in use already( e.g. Reuse of information in loops).
- **Spatial Locality** →
  - ✓ Instructions with addresses close to a recently Executing instruction are likely to be executed soon.
  - ✓ If a word is accessed, adjacent (near) words are likely accessed soon (e.g. Related data items (arrays) are usually stored together; instructions are executed sequentially)
- Cache is a fast small capacity memory that should hold those information which are most likely to be accessed.



# Cache Memory



- Processor issues a Read request, a block of words is transferred from the main memory to the cache, one word at a time.
- Subsequent references to the data in this block of words are found in the cache.
- At any given time, only some blocks in the main memory are held in the cache. Which blocks in the main memory are in the cache is determined by a “Mapping Function”.
- When the cache is full, and a block of words needs to be transferred from the main memory, some block of words in the cache must be replaced. This is determined by a “Replacement Algorithm”.



# Cache Hit

- Existence of a cache is not explicitly known to the processor. The processor issues Read and Write requests in the same manner.
- If the data is in the cache it is called a Read or Write hit.
- Read hit:→
  - The data is obtained from the cache.
- Write hit:→
  - Cache has a replica of the contents of the main memory.
  - Contents of the cache and the main memory may be updated simultaneously. This is the **Write-Through protocol**.
  - Update the contents of the cache, and mark it as updated by setting a bit known as the **Dirty Bit** or **Modified Bit**. The contents of the main memory are updated when this block is replaced. This is **Write-Back or Copy-Back Protocol**.

# Cache Miss

- If the **data is not present** in the cache, then a **Read Miss** or **Write Miss** occurs.
- **Read Miss:**→
  - Block of words containing this requested word is copied from the memory to Cache.
  - After the entire block is transferred, the desired word is forwarded to the processor.  
The desired word may also be forwarded to the processor as soon as it is read from the main memory. This is called **Load-Through** or **Early-Restart**.
- **Write-Miss:**→
  - **Write-Through** protocol is used, then the contents of the main memory are updated directly.
  - If **Write-Back** protocol is used, the block containing the addressed word is first brought into the cache. The desired word is overwritten with new information.

# Performance Of Cache Memory

- All the memory accesses are directed first to Cache
- If the word is in Cache; Access cache to provide it to CPU → **CACHE HIT**
- If the word is not in Cache; Bring a block (or a line) including that word to replace a block now in Cache → **CACHE MISS**
- **Hit Ratio** - % of memory accesses satisfied by Cache memory system
- **Te**: Effective Memory Access Time of the memory system
- **Tc**: Cache Access Time
- **Tm**: Main memory Access Time
- $Te = h * Tc + (1 - h) [Tc + Tm]$
- **Example:**  
 $Tc = 0.4 \mu s, Tm = 1.2 \mu s, h = 85\%$
- $Te = 0.85 * 0.4 + (1 - 0.85) * 1.6 = 0.58 \mu s$

# Cache Coherence Problem

- A bit called as “valid bit” is provided for each block.
- If the block contains valid data, then the bit is set to 1, else it is 0.
- Valid bits are set to 0, when the power is just turned on.
- When a block is loaded into the cache for the first time, the valid bit is set to 1.
- Data transfers between main memory and disk occur directly bypassing the cache.
- When the data on a disk changes, the main memory block is also updated.
- However, if the data is also resident in the cache, then the valid bit is set to 0.

What happens if the data in the disk and main memory changes and the write-back protocol is being used?

- In this case, the data in the cache may also have changed and is indicated by the dirty bit.
- The copies of the data in the cache, and the main memory are different. This is called the **Cache Coherence Problem**.
- One option is to force a write-back before the main memory is updated from the disk.

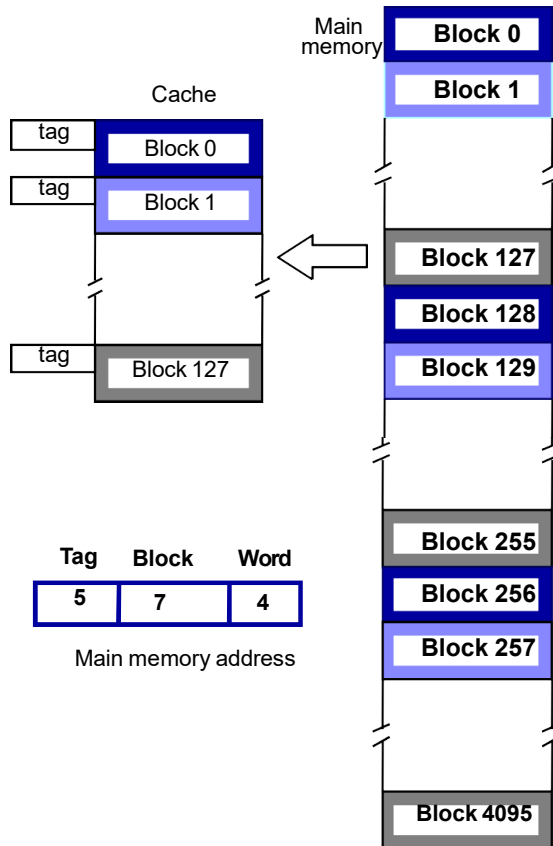
# Cache Memory Mapping Function

- **Specification of correspondence between main memory blocks and cache blocks**
- **Mapping functions determine how memory blocks are placed in the cache.**
- **Three different types of mapping functions:**
  - Direct mapping
  - Associative mapping
  - Set-associative mapping

## **A simple processor example:**

- Cache consisting of 128 blocks of 16 words each.
- Total size of cache is 2048 (2K) words.
- Main memory is addressable by a 16-bit address.
- Main memory has 64K words.
- Main memory has 4K blocks of 16 words each.

# Direct Mapping



Simple to implement but not very flexible.

- Each memory block has **only one place** to load in Cache memory.
- Block  $j$  of the Main Memory maps to  $j$  modulo 128 of the Cache.  
**0 maps to 0, 129 maps to 1.**
- More than one memory block is mapped onto the same position in the cache.
- May lead to contention for cache blocks even if the cache is not full.
- Resolve the contention by allowing new block to replace old block, leading to a trivial replacement algorithm.
  - Low order 4 bits determine one of the 16 words in a block.
  - When a new block is brought into the cache, the next 7 bits determine which cache block this new block is placed in.
  - High order 5 bits determine which of the possible 32 blocks is currently present in the cache. These are tag bits.
- Memory address is divided into three fields:

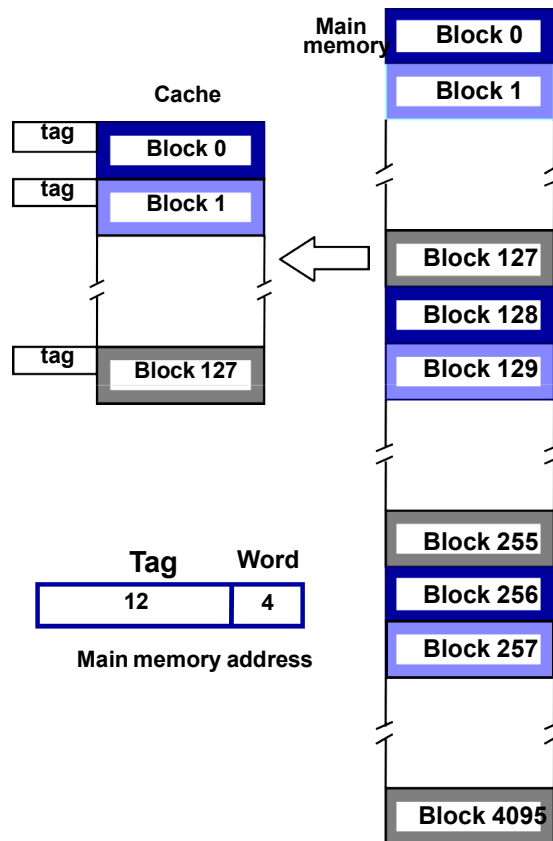
# Direct Mapping

Each memory block has only one place to load in Cache memory.

## Operation

1. As execution proceeds, the 7-bit cache block field of each address generated by the processor points to a particular block location in the cache.
2. The high-order 5 bits of the address are compared with the tag bits associated with that cache location.
3. If they match, then the desired word is in that block of the cache.
4. If there is no match, then the block containing the required word must first be read from the main memory and loaded into the cache.
5. The direct-mapping technique is easy to implement, but it is not very flexible.

# Associative Mapping



1. Main memory block can be placed into any cache position.

2. Memory address is divided into two fields:

✓ Low order 4 bits identify the word within a block.

✓ High order 12 bits or tag bits identify a memory block when it is resident in the cache.

3. The tag bits of an address received from the processor are compared to the tag bits of each block of the cache to see if the desired block is present. This is called the *Associative-Mapping Technique*.

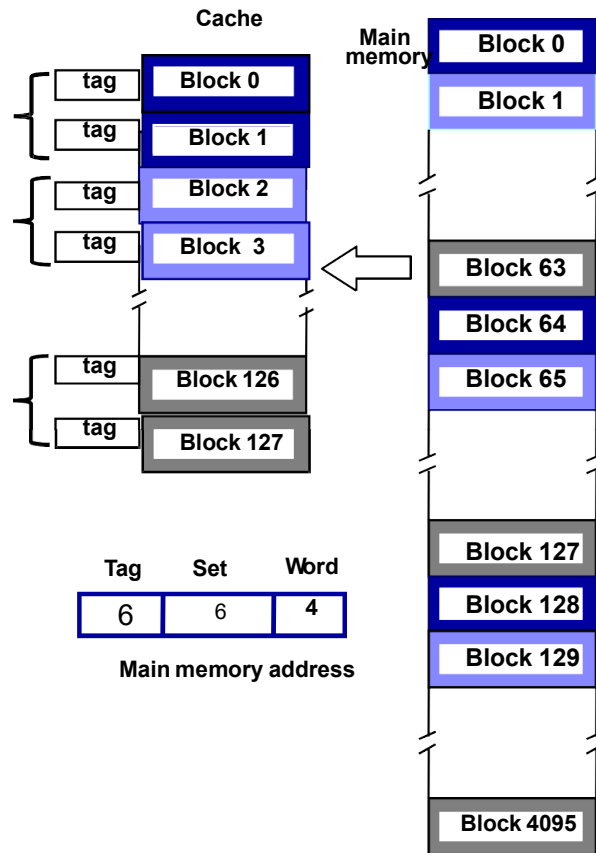
4. Flexible, and uses cache space efficiently.

5. Replacement algorithms can be used to replace an existing block in the cache when the cache is full.

6. Cost is higher than direct-mapped cache because of the need to search all 128 patterns to determine whether a given block is in the cache.



# Set-Associative Mapping



1. Blocks of cache are grouped into sets.

2. Mapping function allows a block of the main memory to reside in any block of a specific set.

3. Divide the cache into 64 sets, with two blocks per set.

4. Memory block 0, 64, 128 etc. map to block 0, and they can occupy either of the two positions.

5. Memory address is divided into three fields:

- 6 bit field determines the set number.

- High order 6 bit fields are compared to the tag fields of the two blocks in a set.

6. Set-associative mapping combination of direct and associative mapping.

7. Number of blocks per set is a design parameter.

- One extreme is to have all the blocks in one set, requiring no set bits (fully associative mapping).

- Other extreme is to have one block per set, is the same as direct mapping.

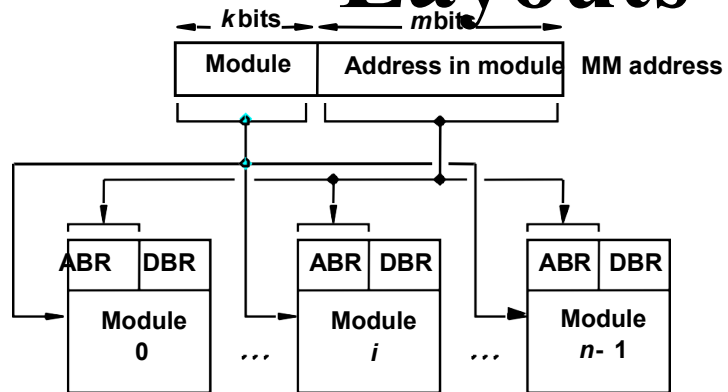
# Performance Considerations

- A key design objective of a computer system is to achieve the best possible performance at the lowest possible cost.
  - Price/performance ratio is a common measure of success.
- Performance of a processor depends on:
  - How fast machine instructions can be brought into the processor for execution.
  - How fast the instructions can be executed.

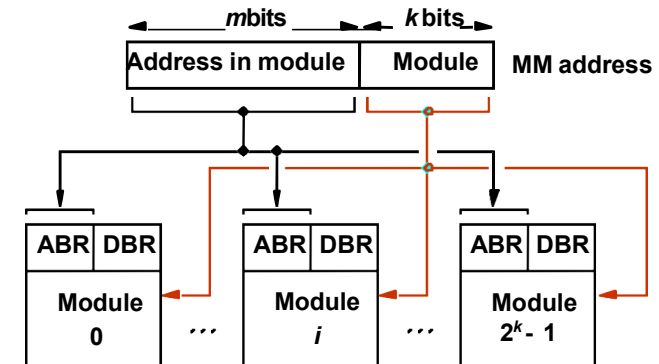
# Memory Interleaving

- Divides the memory system into a number of memory modules.
  - Each module has its own address buffer register (ABR) and data buffer register (DBR).
- Arranges addressing so that successive words in the address space are placed in different modules.
- When requests for memory access involve consecutive addresses, the access will be to different modules.
- Since parallel access to these modules is possible, the average rate of fetching words from the Main Memory can be increased.

# Methods of Address Layouts



- *Consecutive words are placed in a module.*
- *High-order  $k$  bits of a memory address determine the module.*
- *Low-order  $m$  bits of a Memory Address determine the word within a module.*
- *When a block of words is transferred from Main Memory to Cache, only one Module is busy at a time.*



- *Consecutive words are located in consecutive Modules.*
- Consecutive Addresses can be located in consecutive Modules.*
- *While transferring a block of data, several memory modules can be kept busy at the same time.*

# Hit Rate and Miss Penalty

- **Hit Rate**
- **Miss Penalty**
- **Hit Rate can be improved by increasing block size, while keeping Cache size constant**
- **Block sizes that are neither very small nor very large give best results.**
- **Miss Penalty can be reduced if Load-Through approach is used when loading new blocks into Cache.**

# Caches on the Processor Chip

- In high performance processors 2 levels of caches are normally used.
- Avg access time in a system with 2 levels of caches is

$$T_{ave} = h_1c_1 + (1-h_1)h_2c_2 + (1-h_1)(1-h_2)M$$

# Virtual Memories

# Virtual Memories

- *Recall that an important challenge in the design of a computer system is to provide a large, fast memory system at an affordable cost.*
- **Architectural solutions to increase the effective Speed and Size of the Memory System.**
- **Cache memories were developed to increase the effective speed of the Memory System.**
- **Virtual Memory is an architectural solution to increase the EFFECTIVE SIZE of the Memory System.**



## Virtual Memories (contd..)

- *Recall that the addressable memory space depends on the number of address bits in a computer.*
  - For Example, if a computer issues 32-bit addresses, the addressable memory space is 4G bytes.
- Physical Main Memory in a computer is generally not as large as the entire possible addressable space.
  - Physical Memory typically ranges from a few hundred megabytes to 8G bytes.
- Large Programs that cannot fit completely into the Main Memory have their parts stored on Secondary Storage Devices such as magnetic disks.
  - Pieces of programs must be transferred to the main memory from secondary storage before they can be executed.

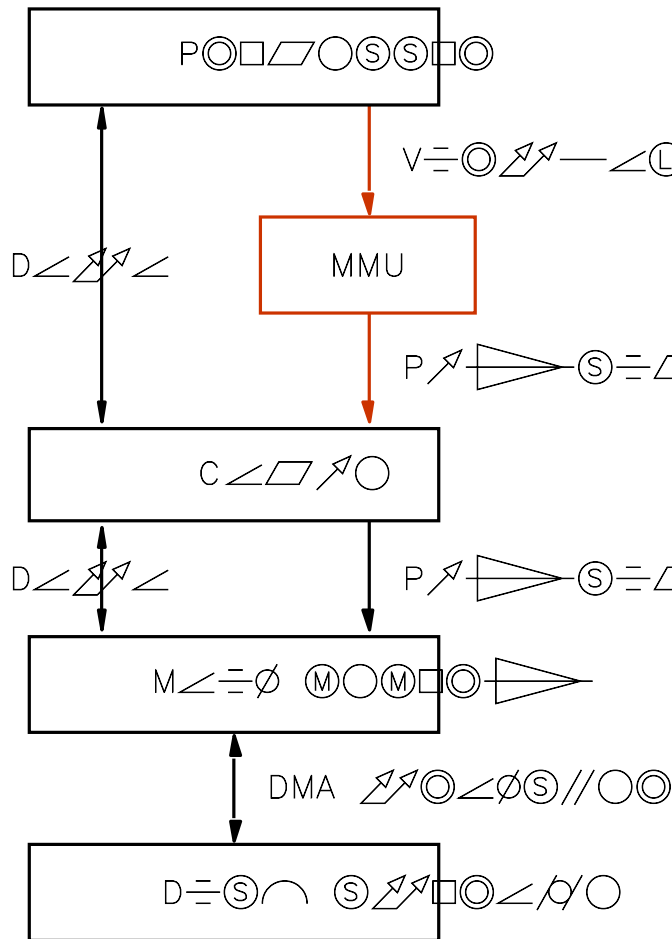
## Virtual Memories (contd..)

- **When a new piece of a program is to be transferred to the Main Memory, and the Main Memory is full, then some other piece in the Main Memory must be replaced.**
  - **Recall, this is very similar to what we studied in case of Cache Memories.**
- *Operating System automatically transfers data between the Main Memory and Secondary Storage.*
  - *Application Programmer need not be concerned with this transfer.*
  - *Also, Application Programmer does not need to be aware of the limitations imposed by the available physical Memory.*

# Virtual Memories (contd..)

- Techniques that automatically move program and data between main memory and secondary storage when they are required for execution are called Virtual-Memory Techniques.
- Programs and processors refer an instruction or data independent of the size of the Main Memory.
- Processor issues binary addresses for Instructions and Data.
  - These binary addresses are called logical or virtual addresses.
- Virtual Addresses are translated into Physical Addresses by a combination of Hardware and Software Subsystems.
  - If virtual address refers to a part of the Program that is currently in the Main Memory, it is accessed immediately.
  - If the address refers to a part of the Program that is not currently in the Main Memory, it is first transferred to the Main Memory before it can be used.

# Virtual memory organization



• *Memory Management Unit (MMU) translates Virtual Addresses into Physical Addresses.*

• *If the desired Data or Instructions are in the Main Memory they are fetched as described previously.*

• *If the desired Data or Instructions are not in the Main Memory, they must be transferred from Secondary Storage to the Main Memory.*

• *MMU causes the Operating System to bring the Data from the Secondary Storage into the Main Memory.*

# Address Translation

- Assume that Program and Data are composed of fixed-length units called Pages.
- A Page consists of a Block of words that occupy contiguous locations in the Main Memory.
- Page is a basic unit of information that is transferred between Secondary Storage and Main Memory.
- Size of a Page commonly ranges from 2K to 16K bytes.
  - *Pages should not be too small, because the access time of a Secondary Storage device is much larger than the Main Memory.*
  - *Pages should not be too large, else a large portion of the Page may not be used, and it will occupy valuable space in the Main Memory.*

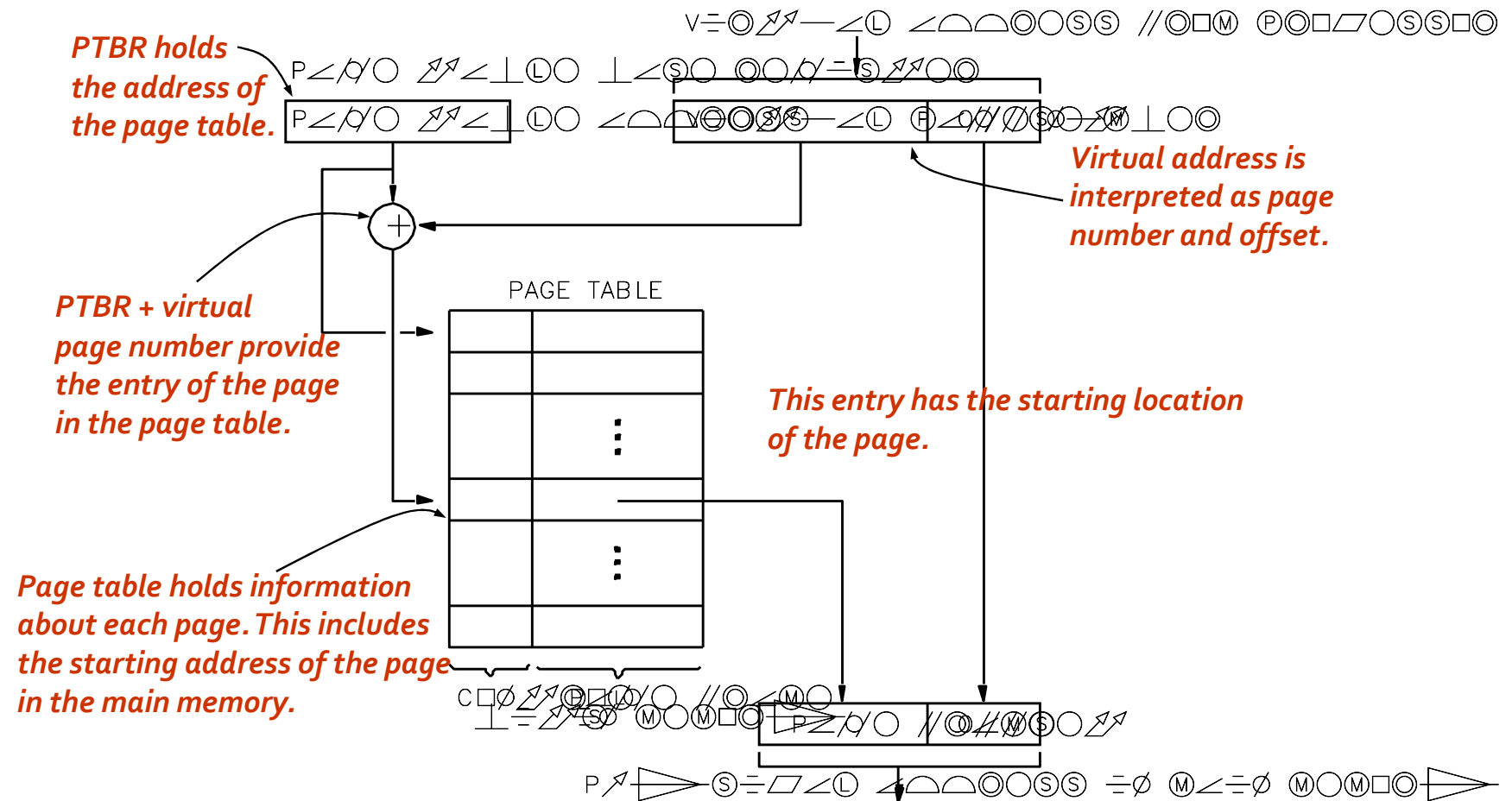
# Address translation (contd..)

- Concepts of virtual memory are similar to the concepts of Cache Memory.
- Cache Memory:
  - Introduced to bridge the speed gap between the processor and the main memory.
  - Implemented in hardware.
- Virtual Memory:
  - Introduced to bridge the speed gap between the main memory and secondary storage.
  - Implemented in part by software.
  - **to increase the EFFECTIVE SIZE of the memory system.**

# Address Translation (contd..)

- Each virtual or Logical Address generated by a processor is interpreted as a Virtual Page Number (high-order bits) plus an Offset (low-order bits) that specifies the location of a particular byte within that page.
- Information about the main memory location of each page is kept in the Page Table.
  - Main memory address where the page is stored.
  - Current status of the page.
- Area of the main memory that can hold a page is called as Page Frame.
- Starting address of the page table is kept in a Page Table Base Register.
- Virtual page number generated by the processor is added to the contents of the page table base register.
  - This provides the address of the corresponding entry in the page table.
- The contents of this location in the page table give the starting address of the page if the page is currently in the main memory.

# Address Translation (contd..)





# Address Translation (contd..)

- Page table entry for a page also includes some control bits which describe the status of the page while it is in the main memory.
  - One bit indicates the validity of the page.
  - Indicates whether the page is actually loaded into the main memory.
  - Allows the operating system to invalidate the page without actually removing it.
- One bit indicates whether the page has been modified during its residency in the main memory.
  - This bit determines whether the page should be written back to the disk when it is removed from the main memory.
  - Similar to the dirty or modified bit in case of cache memory.

# Address Translation (contd..)

- **Where should the page table be located?**
- **Recall that the page table is used by the MMU for every read and write access to the memory.**
  - Ideal location for the page table is within the MMU.
- **Page table is quite large.**
- **MMU is implemented as part of the processor chip.**
- **Impossible to include a complete page table on the chip.**
- **Page table is kept in the main memory.**
- **A copy of a small portion of the page table can be accommodated within the MMU.**
  - Portion consists of page table entries that correspond to the most recently accessed pages.

# Address Translation (contd..)

- **A small cache called as Translation Lookaside Buffer (TLB) is included in the MMU.**
  - TLB holds page table entries of the most recently accessed pages.
- **Recall that cache holds most recently accessed blocks from the main memory.**
  - Operation of the TLB and page table in the main memory is similar to the operation of the cache and main memory.
- **Page table entry for a page includes:**
  - Address of the page frame where the page resides in the main memory.
  - Some control bits.
- **In addition to the above for each page, TLB must hold the virtual page number for each page.**

# Address Translation (contd..)

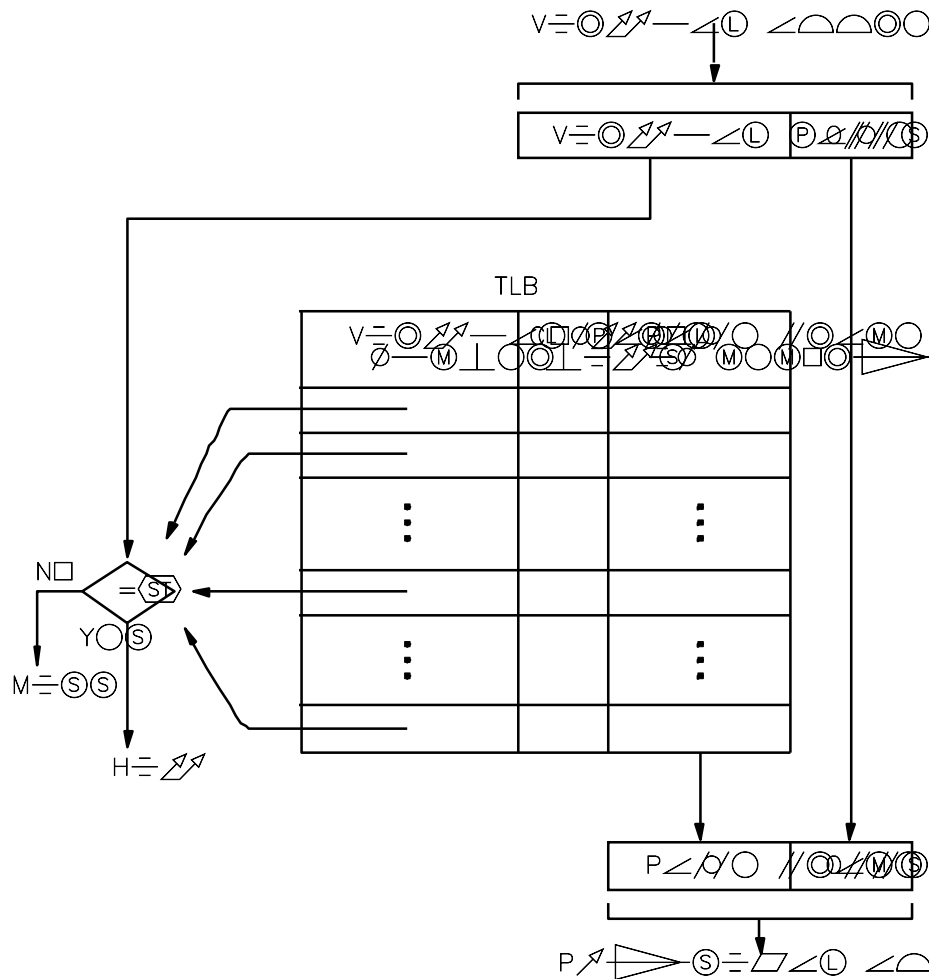
## Associative-mapped TLB

*High-order bits of the virtual address generated by the processor select the virtual page.*

*These bits are compared to the virtual page numbers in the TLB.*

*If there is a match, a hit occurs and the corresponding address of the page frame is read.*

*If there is no match, a miss occurs and the page table within the main memory must be consulted.*

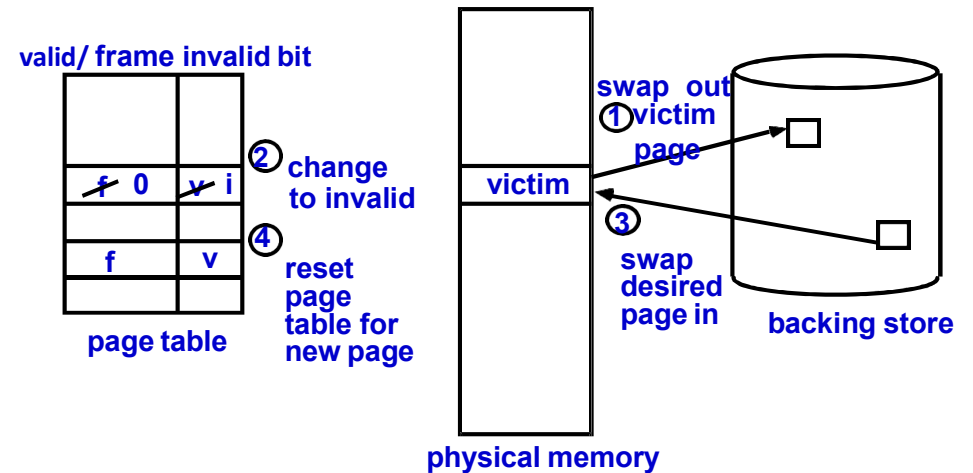


*Set-associative mapped TLBs are found in commercial processors.*

# **PAGE REPLACEMENT**

# PAGE REPLACEMENT

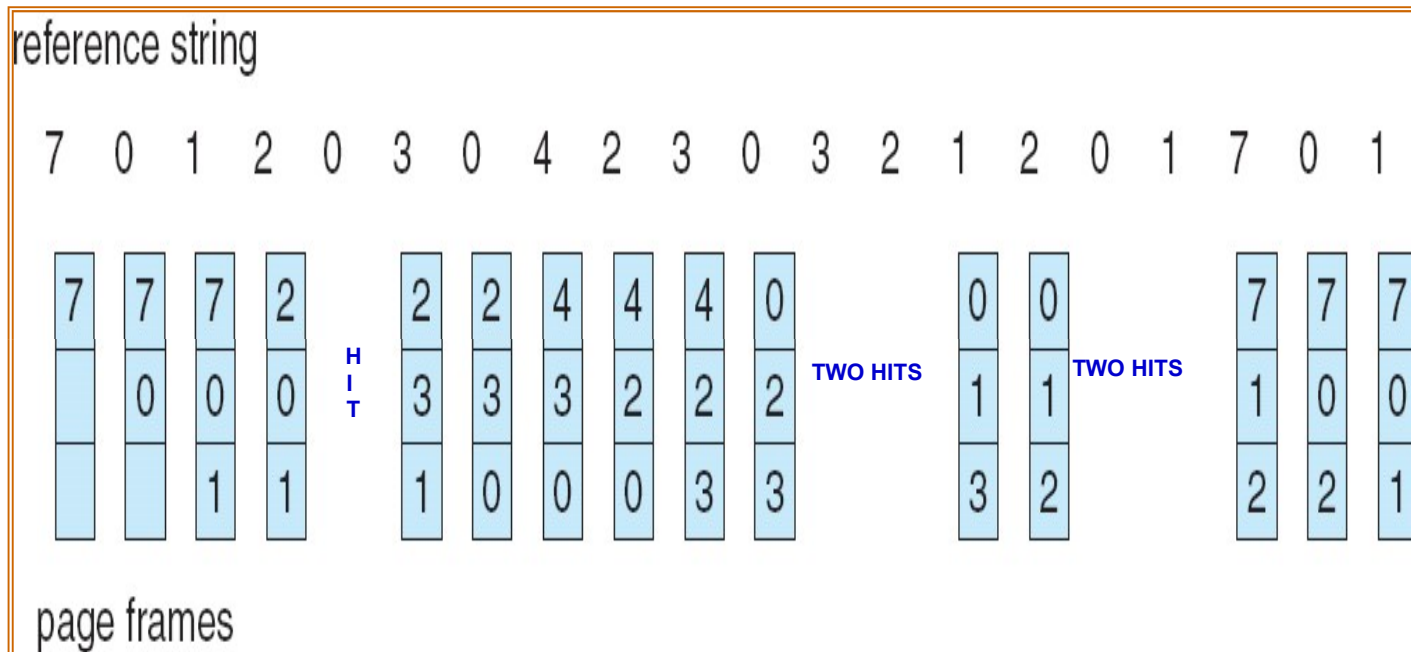
- **Decision on which page to displace to make room for an incoming page when no free frame is available**
- Page fault service routine
  1. Find the location of the desired page on the backing store
  2. Find a free frame
    - *If there is a free frame, use it*
    - *Otherwise, use a Page-Replacement algorithm to select a victim frame*
    - *Write the victim page to the backing store*
  3. Read the desired page into the (newly) free frame
  4. Restart the user process



# First-In-First-Out (FIFO) Algorithm

- **Replacement is depends upon the arrival time of a page to memory.**
- **A page is replaced when it is oldest (in the ascending order of page arrival time to memory).**
- **As it is a FIFO queue no need to record the arrival time of a page & the page at the head of the queue is replaced.**
- **Performance is not good always**
- **When a active page is replaced to bring a new page, then a page fault occurs immediately to retrieve the active page.**
- **To get the active page back some other page has to be replaced. Hence the page fault increases.**

# FIFO Page Replacement



**15 PAGE FAULTS**



# Problem with FIFO Algorithm

- Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- 3 frames (3 pages can be in memory at a time per process)

|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 4 | 5 |
| 2 | 2 | 1 | 3 |
| 3 | 3 | 2 | 4 |

9 page faults

- 4 frames

|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 5 | 4 |
| 2 | 2 | 1 | 5 |
| 3 | 3 | 2 |   |
| 4 | 4 | 3 |   |

Page fault increases

10 page faults

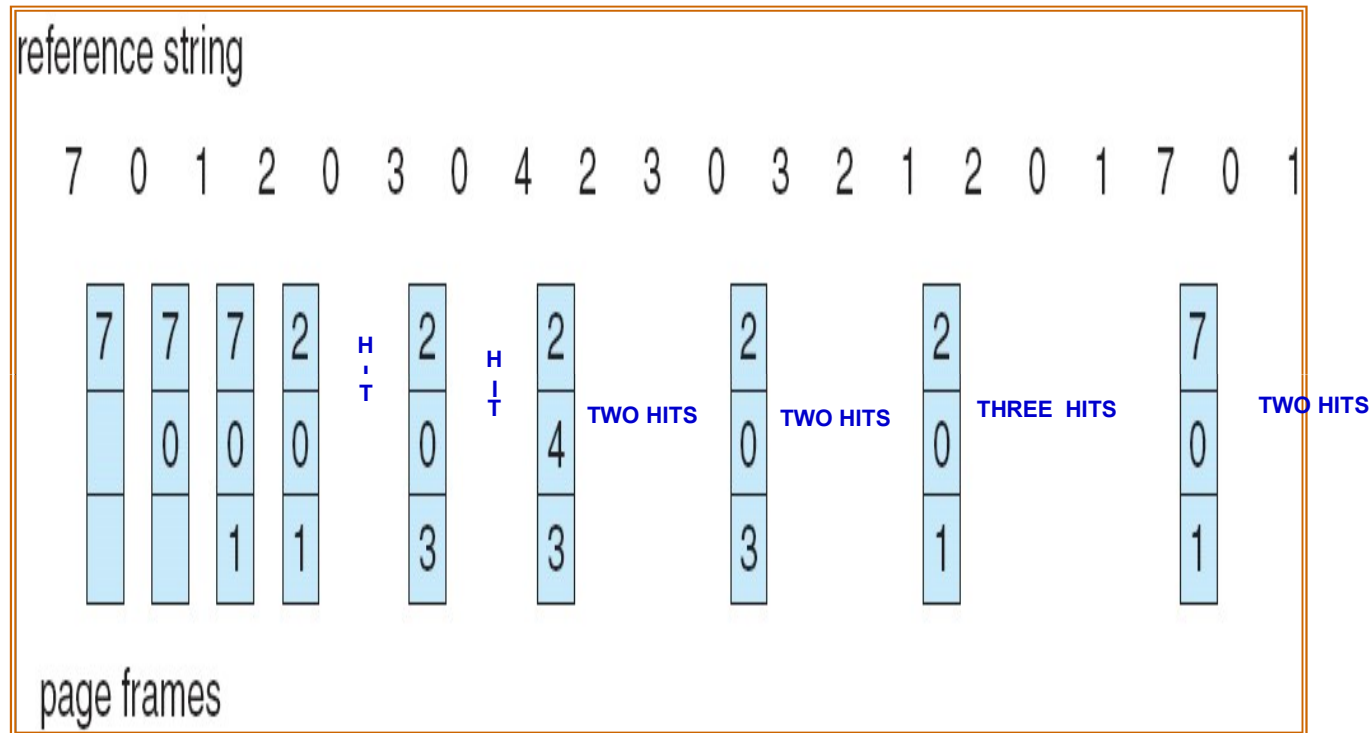
UNEXPECTED

- **Belady's Anomaly**:  $\rightarrow$  more frames  $\Rightarrow$  more page faults

# Optimal Algorithm

- To recover from belady's anomaly problem :→ **Use Optimal page replacement algorithm**
- **Replace the page that will not be used for longest period of time.**
- This guarantees lowest possible page fault rate for a fixed number of frames.
- **Example :→**
  - First we found 3 page faults to fill the frames.
  - Then replace page 7 with page 2 because it will not needed up to the 18<sup>th</sup> place in reference string.
  - Finally there are 09 page faults.
  - Hence it is better than FIFO algorithm (15 page Faults).

# Optimal Page Replacement



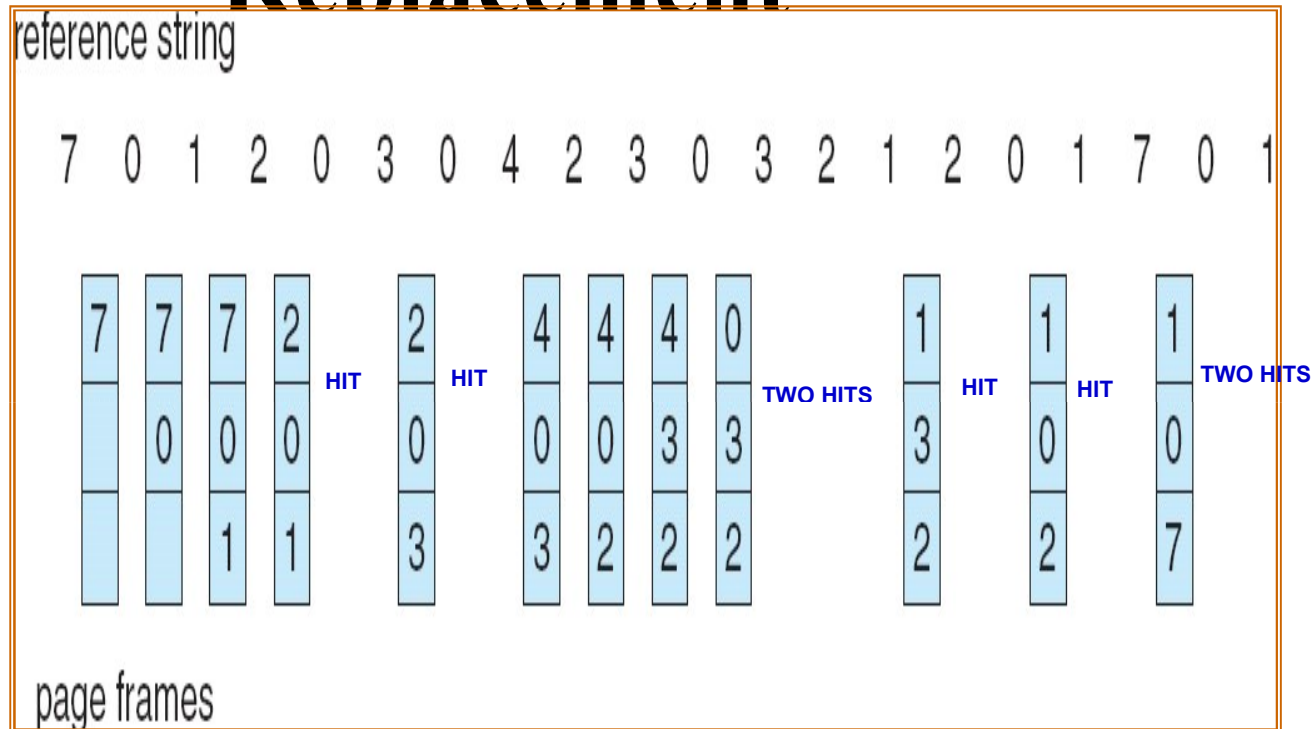
- **Difficulty with Optimal Algorithm :: It always needs future knowledge of reference string.**

**09 PAGE FAULTS**

# Least Recently Used (LRU) Algorithm

- LRU algorithm lies between FIFO & Optimal algorithm ( in terms of page faults).
- FIFO :→ time when page brought into memory.
- OPTIMAL :→ time when a page will used.
- **LRU** :→ Use the recent past as an approximation of near future (so it cant be replaced), then we will replace that page which has not been used for longest period of time. Hence it is least recently used algorithm.
- **Example** :→
  - Up to 5<sup>th</sup> page fault it is same as optimal algorithm.
  - When page 4 occur LRU chose page 2 for replacement.
  - Here we find only 12 page faults.

# LRU Page Replacement



**12 PAGE FAULTS**

# Least Recently Used (LRU) Algorithm

- Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 5 |
| 2 | 2 | 2 | 2 | 2 |
| 3 | 5 | 5 | 4 | 4 |
| 4 | 4 | 3 | 3 | 3 |

- Counter implementation
  - Every page entry has a counter; every time page is referenced through this entry, copy the clock into the counter
  - When a page needs to be changed, look at the counters to determine which are to change

# LFU :: Least Frequently Used

The block that was used for less no of times, must be the one to be replaced first

A counter can be used to record frequency.

In case of a tie use FIFO

**Blue Color are HITS**

| 1 | 3 | 4 | 3 | 1 | 5 | 2 | 1 | 1 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
|   | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 |
|   |   | 4 | 4 | 4 | 5 | 2 | 2 | 2 | 2 | 4 | 5 |

# LIFO :: Last In First Out

The block that was brought last, must be the one to be replaced first.  
It can be implemented by using a Stack, i.e. top of the stack to be replaced.

**Blue Color are HITS**

| 1 | 3 | 4 | 3 | 1 | 5 | 2 | 1 | 1 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
|   | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 |
|   |   | 4 | 4 | 4 | 5 | 2 | 2 | 2 | 2 | 4 | 5 |



# MFU :: Most Frequently Used

The Block that was used for maximum no of times, must be the one to be replaced first.

A counter can be used to record frequency

In case of a tie use FIFO

**Blue Color are HITS**

| 1 | 3 | 4 | 3 | 1 | 5 | 2 | 1 | 1 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 5 | 5 | 5 | 5 | 5 | 4 | 1 |
|   | 3 | 3 | 3 | 3 | 3 | 2 | 2 | 2 | 2 | 2 | 5 |
|   |   | 4 | 4 | 4 | 4 | 4 | 1 | 1 | 3 | 3 | 3 |

# MRU :: Most Recently Used

The block that was used in recent past, will be the one to be replaced first.

A timer can be used to record timing

**Blue Color are HITS**

| 1 | 3 | 4 | 3 | 1 | 5 | 2 | 1 | 1 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 5 | 2 | 1 | 1 | 1 | 1 | 1 |
|   | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 |
|   |   | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 5 |